

DISPENSA DELLA LEZIONE

LocalStorage in JavaScript

Salvare, leggere e cancellare dati nel browser.



CAPITOLO 1

Cos'è il localStorage

NOTA

Il localStorage è uno spazio di memorizzazione del browser che salva dati anche dopo la chiusura della pagina. I dati restano disponibili finché non vengono cancellati manualmente.

Lavora con coppie chiave-valore: ogni dato ha una chiave che lo identifica e un valore che lo contiene. Nel localStorage si salvano solo stringhe. Se vuoi memorizzare numeri, booleani, oggetti o array, devi prima trasformarli in testo e poi riconvertirli quando li leggi.

È importante sapere che i dati nel localStorage sono visibili e modificabili direttamente dall'utente tramite il pannello DevTools del browser, nella scheda **Application**. Chiunque può aprire gli strumenti di sviluppo e leggere, cambiare o cancellare i valori salvati. Per questo motivo il localStorage è adatto a conservare solo informazioni non sensibili: preferenze dell'utente (tema grafico, lingua, dimensione del testo), lo stato di un carrello in un e-commerce, l'ultima pagina visitata o impostazioni di configurazione dell'interfaccia. Non deve mai contenere password, token di autenticazione o dati personali riservati.

CAPITOLO 2

setItem

NOTA

`setItem()` aggiunge un dato oppure aggiorna un dato già esistente. Vuole sempre due argomenti: la chiave e il valore.

`setItem()` è il metodo con cui inserisci un dato nel `localStorage`. Se la chiave esiste già, il valore precedente viene sostituito con quello nuovo.

Questo metodo è utile quando vuoi salvare una scelta dell'utente, come una preferenza grafica o la lingua selezionata.

Esempio

```
localStorage.setItem("tema", "scuro");  
localStorage.setItem("lingua", "it");
```

CAPITOLO 3

getItem

NOTA

`getItem()` legge il valore associato a una chiave. Se la chiave non esiste, restituisce `null`.

`getItem()` serve per recuperare un dato salvato in precedenza. Si usa quando vuoi riutilizzare un'informazione già memorizzata, per esempio per mostrarla a schermo o per fare un controllo.

Il valore restituito è sempre una stringa o `null`. Se ti serve un numero, devi convertirlo con `Number()` prima di usarlo in un calcolo.

Esempio

```
localStorage.setItem("lingua", "it");  
const lingua = localStorage.getItem("lingua");  
console.log(lingua);
```

CAPITOLO 4

removeItem

NOTA

`removeItem()` elimina solo una chiave specifica dal `localStorage`.

`removeItem()` è il metodo da usare quando vuoi cancellare un solo dato salvato. Si indica la chiave da eliminare e il browser rimuove solo quella coppia chiave-valore, lasciando intatto tutto il resto.

È utile quando un utente vuole annullare una scelta, resettare una preferenza o eliminare un'informazione non più necessaria.

Esempio

```
localStorage.setItem("tema", "scuro");  
localStorage.removeItem("tema");
```

CAPITOLO 5

clear

NOTA

`clear()` svuota completamente il localStorage del sito.

`clear()` cancella tutti i dati salvati nel localStorage. Va usato con attenzione, perché rimuove ogni chiave presente, non solo una singola informazione.

Questo metodo è utile quando vuoi ripartire da zero, per esempio in una schermata di logout o in un pulsante che elimina tutte le preferenze memorizzate.

Esempio

```
localStorage.setItem("tema", "scuro");  
localStorage.setItem("lingua", "it");  
localStorage.clear();
```

CAPITOLO 6

JSON.stringify

NOTA

JSON.stringify() trasforma un oggetto o un array in una stringa testuale in formato JSON.

JSON è un formato standard per rappresentare dati strutturati come testo. Poiché il localStorage accetta solo stringhe, **JSON.stringify()** è il passaggio necessario per poter salvare oggetti e array. Il risultato è una stringa che contiene tutti i dati in un formato leggibile e riconvertibile.

Il metodo si usa passando il dato da trasformare come argomento. Il valore originale non viene modificato: viene creata una nuova stringa.

Esempio

```
const studente = {
  nome: "Luca",
  eta: 16
};

const studenteJSON = JSON.stringify(studente);
console.log(studenteJSON);
// risultato: '{"nome":"Luca","eta":16}'

const corsi = ["HTML", "CSS", "JavaScript"];
const corsiJSON = JSON.stringify(corsi);
console.log(corsiJSON);
// risultato: '["HTML","CSS","JavaScript"]'
```

CAPITOLO 7

JSON.parse

NOTA

`JSON.parse()` trasforma una stringa JSON in un oggetto o in un array.

Quando leggi dal `localStorage` un valore salvato con `JSON.stringify()`, ottieni una stringa. Con `JSON.parse()` puoi riconvertire quella stringa nel tipo di dato originale, così da poter accedere di nuovo alle proprietà di un oggetto o agli elementi di un array.

I due metodi lavorano sempre in coppia: `JSON.stringify()` si usa prima di salvare, `JSON.parse()` si usa dopo aver letto.

Esempio

```
const studenteJSON = '{"nome":"Luca","eta":16}';
const studente = JSON.parse(studenteJSON);
console.log(studente.nome);
// risultato: Luca

const corsiJSON = '["HTML","CSS","JavaScript"]';
const corsi = JSON.parse(corsiJSON);
console.log(corsi[0]);
// risultato: HTML
```

CAPITOLO 8

Oggetti e array nel localStorage

NOTA

Oggetti e array si salvano nel localStorage con `JSON.stringify()` e si recuperano con `JSON.parse()`.

Un oggetto o un array non può essere salvato direttamente nel localStorage, perché il localStorage accetta solo stringhe. Il flusso corretto è: trasformare con `JSON.stringify()`, salvare con `setItem()`, leggere con `getItem()` e infine riconvertire con `JSON.parse()`.

Questo approccio è utile quando devi memorizzare strutture più ricche, come i prodotti nel carrello di un e-commerce, le preferenze di configurazione di un'interfaccia o un elenco di voci selezionate dall'utente.

Esempio

```
const studente = {
  nome: "Luca",
  eta: 16
};

const listaCorsi = ["HTML", "CSS", "JavaScript"];

localStorage.setItem("studente", JSON.stringify(studente));
localStorage.setItem("corsi", JSON.stringify(listaCorsi));

const studenteSalvato = JSON.parse(localStorage.getItem("studente"));
const corsiSalvati = JSON.parse(localStorage.getItem("corsi"));

console.log(studenteSalvato);
console.log(corsiSalvati);
```